

Data-Driven Personalised Educational Content Recommendation

Preliminary Project Report — CM3070 Final Project

Vinit Tiwari (Student ID 220174440)

Source code: <https://github.com/VinitTiwari2002/EducationalContentRecommender>

29 June 2026

Contents

1	Introduction	2
1.1	Project Concept	2
1.2	Motivation	2
1.3	Aims and Objectives	2
1.4	Scope and Non-Goals	3
1.5	Contribution	3
1.6	Report Structure	3
2	Literature Review	3
2.1	Scope and Research Questions	3
2.2	Collaborative Filtering in Education: Thai-Nghe et al. (2010)	4
2.3	Modelling Knowledge State: Corbett & Anderson (1995); Piech et al. (2015)	4
2.4	Content-Based Recommendation for MOOCs: Bousbahi & Chorfi (2015)	5
2.5	Technical Foundations	5
2.6	Critical Synthesis and Identified Gap	6
3	Design	7
3.1	System Overview	7
3.2	Data Design	8
3.3	Recommendation Process — Worked Example	8
3.4	Model Design	10
3.5	Evaluation Strategy	10
3.6	Ethics, DEI, and Risks	11
4	Feature Prototype	11
4.1	What the Prototype Implements	11
4.2	Design Decisions and Justification	12
4.3	Results	13
4.4	Evaluation of the Prototype	13
4.5	Limitations and Planned Improvements	14
	References	14

1 Introduction

1.1 Project Concept

For my final project, I am building a hybrid recommendation engine that suggests learning resources to individual learners. The engine combines three signals: how similar learners engaged with content (collaborative filtering), the characteristics of the resources themselves (feature-based content filtering), and the assessment outcomes that followed the engagement. I implement the system as a Python pipeline with a FastAPI service for recommendation queries and a Streamlit dashboard for inspection. I evaluate it offline on the Open University Learning Analytics Dataset (OULAD), which contains 32,593 students, over 10 million Virtual Learning Environment (VLE) interactions, and linked assessment results across seven distance-learning courses.

1.2 Motivation

Online learning has expanded the supply of educational content by orders of magnitude, but learners still have no real way to decide which resource to study next. This is the classical paradox of choice, and in education it has measurable consequences. Most commercial recommendation systems optimise for engagement, which makes sense in retail and media where consumption is the goal. Education is different: I do not want a learner to engage with a resource, I want them to learn from it. A video that holds attention is not the same as a video that closes a knowledge gap.

OULAD makes this gap visible. When I looked at the 23,326 students for whom both VLE logs and assessment results exist, I found a Pearson correlation of $r = 0.274$ between total clicks and mean assessment score. This is a moderate positive association, consistent with the wider educational-analytics literature, and large enough to be useful as a signal. Students who access certain resource types at certain points in a course perform measurably better on assessments. The opportunity, then, is to recommend for *outcome* rather than engagement: to surface the resources that have historically helped learners with similar profiles improve their scores. This turns the recommendation problem from a popularity-and-similarity question into an outcome-conditioned one, and it is the central contribution I intend to make.

1.3 Aims and Objectives

Aim. To design, build, and rigorously evaluate a hybrid educational content recommendation engine that combines collaborative filtering, feature-based content filtering, and learning-outcome signals, and to show that the hybrid outperforms its individual components on OULAD.

Objectives.

1. **Data foundation.** Acquire, clean, and characterise OULAD, producing a reproducible preprocessing pipeline that yields a user-item interaction matrix, item-level feature vectors, and a temporal train/test split with no leakage.
2. **Model progression.** Build a graded series of recommenders — Random, Popularity, Collaborative (truncated SVD), Content-Based (cosine similarity over item features), and Hybrid (weighted ensemble) — behind a common interface, so that comparisons across them are like-for-like.
3. **Outcome-aware evaluation.** Score every model with standard ranking metrics (Precision@10, Recall@10, NDCG@10) and an outcome-weighted variant that rewards recommendations correlated with higher assessment scores. Use cross-validation, an ablation study, and paired statistical testing.

4. **Working artefact.** Deliver a FastAPI endpoint and Streamlit dashboard that allow a marker to query the recommender for any student in the test set and inspect both the recommendations and the reasons behind them.

1.4 Scope and Non-Goals

My project is offline and observational. I do not deploy to live learners, run online A/B tests, or perform any intervention study. My outcome signal is therefore a proxy: I measure whether recommendations are *associated with* better outcomes in held-out historical data, not whether issuing those recommendations *causes* better outcomes. I acknowledge this limitation throughout, and the evaluation chapter discusses it explicitly.

I also do not attempt full deep knowledge tracing in the sense of Piech et al. (2015) — that is a sub-project in its own right. Instead, I take the *insight* from knowledge tracing — that prior assessment performance is informative about current need — and use assessment scores as a feature in my hybrid model.

I work on a single dataset. OULAD is rich enough to demonstrate every technique, and adding a second dataset would dilute the depth of evaluation rather than strengthen it. Cross-dataset generalisation is a future extension.

1.5 Contribution

Existing research in educational recommendation tends to specialise. Collaborative methods (Thai-Nghe et al., 2010) ignore content features. Content-based methods (Bousbahi & Chorfi, 2015) ignore collaborative signal. Knowledge-tracing approaches (Corbett & Anderson, 1995; Piech et al., 2015) model knowledge state but stop short of recommending resources. To my knowledge, no published study combines all three — collaborative, content, and outcome — and evaluates the combination on a public dataset with an outcome-aware metric. That is the gap I want to occupy. I am not inventing a new algorithm; my contribution is the systematic, reproducible combination and outcome-aware evaluation of established techniques on a publicly available educational dataset.

1.6 Report Structure

The rest of this report is organised as follows. **Chapter 2** reviews the most relevant prior work and the technical literature on matrix factorisation and recommender evaluation, and pins down the gap I am addressing. **Chapter 3** sets out the system design: data, models, evaluation strategy, and the ethical and DEI considerations specific to educational data. **Chapter 4** describes what I have built by the time of this submission — a working data-loading and preprocessing pipeline with two baseline recommenders and a full evaluation framework — together with results, an evaluation of those results, and the planned path from prototype to final system.

2 Literature Review

2.1 Scope and Research Questions

I focus this review on the intersection of recommender systems and educational technology, and on four issues my project must address: whether collaborative filtering techniques developed for commercial domains carry over to education, how content features can substitute for or complement collaborative signal when interaction data is sparse, how prior work has used

learning outcomes as a recommendation signal, and what evaluation methodology fits an offline educational recommender on a public dataset. Three research questions structure the review:

1. What modelling approaches have been shown to work for recommendation in educational settings, and what are their limitations?
2. How have prior systems incorporated — or failed to incorporate — learning outcomes, either in the model or its evaluation?
3. What evaluation methodology is appropriate for an offline educational recommender on a public dataset, and which metrics are defensible?

I examine four primary works in detail, each addressing one or more of these questions, and then draw methodological inferences from three supporting technical references on matrix factorisation, evaluation, and ranking. The synthesis at the end identifies the gap that motivates my project.

2.2 Collaborative Filtering in Education: Thai-Nghe et al. (2010)

Thai-Nghe et al. (2010) were among the first to apply matrix factorisation to educational data. Their study reframes student performance prediction as a recommender-system problem: students are the users, tasks are the items, and the score a student achieves on a task is the implicit rating. They show that matrix factorisation beats traditional regression on three educational datasets, including KDD Cup 2010 Algebra and Bridge to Algebra.

The contribution matters because it shows that the algebra of collaborative filtering is not domain-specific. Latent-factor models can capture student and task abilities without the textual or behavioural metadata used in commercial systems.

The limitations bear directly on my project. First, the system *predicts* — it estimates how a student would score on a known task — but it does not *recommend* unseen resources for the student to study next. Prediction tells the learner where they stand; recommendation tells them where to go. Second, the model treats every task as an opaque identifier, with no notion of difficulty, topic, or content. The result is a purely collaborative model that suffers the classical cold-start problem for new tasks and new students. Third, the evaluation is RMSE on held-out scores; whether following the predictions would have *improved* the student’s learning outcome is not addressed and, given the data, could not have been. For my project, Thai-Nghe et al. establish that matrix factorisation works in education — which is why I include an SVD model in my progression — but they also show why a pure collaborative approach is not enough.

2.3 Modelling Knowledge State: Corbett & Anderson (1995); Piech et al. (2015)

A parallel strand of research treats education not as a recommendation problem at all, but as a knowledge-state estimation problem. The seminal contribution is Corbett & Anderson’s (1995) Bayesian Knowledge Tracing (BKT). BKT models student knowledge as a hidden state, updated after each observed practice item according to four parameters: the prior probability of knowing a skill, the probability of moving from “not known” to “known” given a learning opportunity, and the slip and guess probabilities relating knowledge state to observed performance. The model is principled, interpretable, and the empirical workhorse of the Cognitive Tutor family of intelligent tutoring systems.

Piech et al. (2015) introduced Deep Knowledge Tracing (DKT). DKT replaces the BKT Markov model with a recurrent neural network (LSTM) that learns its own latent knowledge representation from raw sequences of student responses. DKT beats BKT on standard benchmarks,

largely because it sheds the assumption that skills are independent: an LSTM can capture cross-skill transfer effects that BKT cannot.

Two things from this strand are useful to me. First, prior assessment performance is genuinely informative about learner state and should not be discarded — which justifies my use of assessment scores as a feature in the hybrid model. Second, and more importantly, both papers stop at *knowledge estimation*; neither says what to recommend next. BKT, deployed in tutoring systems, typically uses hand-authored progression rules over the estimated state; DKT is purely evaluative. Both are also *content-blind*: they model student-skill interaction without any representation of the resource the student engaged with, so they cannot distinguish “watch the video” from “attempt the quiz” even though the two interventions can have very different effects. DKT has also been repeatedly criticised on evaluation — Wilson et al. (2016) showed that simpler models match its claimed performance under fairer evaluation protocols. The lesson I take from this is that any deep-learning component must be evaluated against simpler baselines, not in isolation.

2.4 Content-Based Recommendation for MOOCs: Bousbahi & Chorfi (2015)

Bousbahi & Chorfi (2015) take the opposite stance to Thai-Nghe et al. Their MOOC-Rec system represents each course by a vector of metadata features (topic, level, language, prerequisites, duration) and recommends courses whose feature vector is closest to a learner’s stated profile. The approach is case-based: a new query is matched to the closest historical case, and the recommendation is explainable because the matching dimensions can be surfaced to the learner.

The strengths are real and complement Thai-Nghe et al. directly. Content-based methods do not need historical interaction data and so handle cold-start gracefully, and they are auditable: a recommender can explain *why* it suggested a resource by naming the features that contributed most. In education, where unexplained algorithmic recommendations can erode learner trust and where new courses and new learners arrive frequently, both properties matter.

The limitations are also real. First, there is no collaborative component, so the system cannot learn from what worked for similar learners. If learners with profile P historically did better with resource A than resource B , MOOC-Rec has no way to discover this. Second, the evaluation is standard information-retrieval metrics on a small expert-curated test set; there is no measurement of whether the recommendations led to better learning. Third, the feature representation is essentially textual metadata and ignores behavioural features such as historical engagement patterns or the outcome correlations of the resource. For my project, Bousbahi & Chorfi justify the content-based component of the hybrid — particularly for cold-start — but also show why content-based alone is not enough.

2.5 Technical Foundations

Koren, Bell & Volinsky (2009) set out the algebra and optimisation of SVD-based recommendation. I plan to use truncated SVD, following their formulations, with regularisation and biased baseline terms as they recommend.

Ricci, Rokach & Shapira (2015) cover evaluation methodology for recommender systems in depth. Their treatment of offline evaluation — choice of metric, construction of train/test splits, trade-offs between precision-style and recall-style metrics — underpins my evaluation strategy in Chapter 3 of this report. The most important point I take from them is that temporal splits (train on past, test on future) are essential for any recommender that will be deployed sequentially;

random splits systematically overestimate performance because they let future information leak into training.

Rendle et al. (2012) address a problem that is central to me: the OULAD data is *implicit* (the learner accessed a resource) rather than *explicit* (the learner rated a resource). Treating absence-of-interaction as negative evidence is the well-known trap of implicit-feedback recommenders, and BPR’s pairwise-ranking formulation is the standard mitigation. My evaluation framework follows their lead in treating ranking quality (NDCG) as the primary success criterion rather than rating-prediction accuracy.

2.6 Critical Synthesis and Identified Gap

Mapping the four primary works against the three research questions makes the pattern clear.

Work	Modelling approach	Uses content features	Uses outcomes	Evaluation focus
Thai-Nghe et al. (2010)	Collaborative (MF)	No	Outcome-as-target	RMSE prediction
Corbett & Anderson (1995)	Knowledge tracing (HMM)	No	Outcome-as-target	Knowledge estimation
Piech et al. (2015)	Knowledge tracing (RNN)	No	Outcome-as-target	Next-answer prediction
Bousbah & Chorfi (2015)	Content-based	Yes	No	IR metrics
My project	Hybrid (CF + content)	Yes	Outcome-as-evaluation signal	Ranking + outcome-weighted

Two specific gaps come out of this. The first is *architectural*: no published work combines collaborative filtering, content features, and an outcome signal in a single recommender evaluated on a public educational dataset. Each strand of research has stayed inside its own paradigm. The second is *methodological*: where outcomes are used at all, they are used as the *target* of prediction (Thai-Nghe et al.; Piech et al.), not as a *quality signal for recommendation evaluation*. The distinction matters. Predicting that a learner will fail an assessment is not the same as recommending content that will reduce the probability of failure; only the second is actionable for the learner.

There is one further methodological concern, taken from Wilson et al.’s critique of DKT and from Ricci et al.’s argument for temporal splits: any system claiming improvement over baselines has to be evaluated against carefully chosen baselines under a protocol that does not allow leakage. I address this directly with a graded model progression (Random → Popularity → SVD → Content → Hybrid) under a temporal 80/20 split, paired statistical testing across folds, and an ablation study to isolate the contribution of each component.

I am therefore positioning my contribution not as a new algorithm but as a *systematic combination* — collaborative filtering, feature-based content matching, and outcome-aware evaluation — implemented and evaluated end-to-end on a public dataset, with enough methodological rigour to allow direct comparison against the prior work surveyed here.

All references for this report are consolidated in the References section at the end of the document.

3 Design

3.1 System Overview

I have designed the system as a five-stage offline data pipeline. Its output is a ranked recommendation list and an evaluation report, fronted by a FastAPI service for interactive querying and a Streamlit dashboard for inspection. The five stages — ingestion, preprocessing, feature engineering, modelling, and evaluation — are organised as independent components, each consuming the output of the previous stage and persisting its own artefacts for downstream reuse. Figure 3.1 shows the high-level flow.

Figure 3.1 — Pipeline architecture

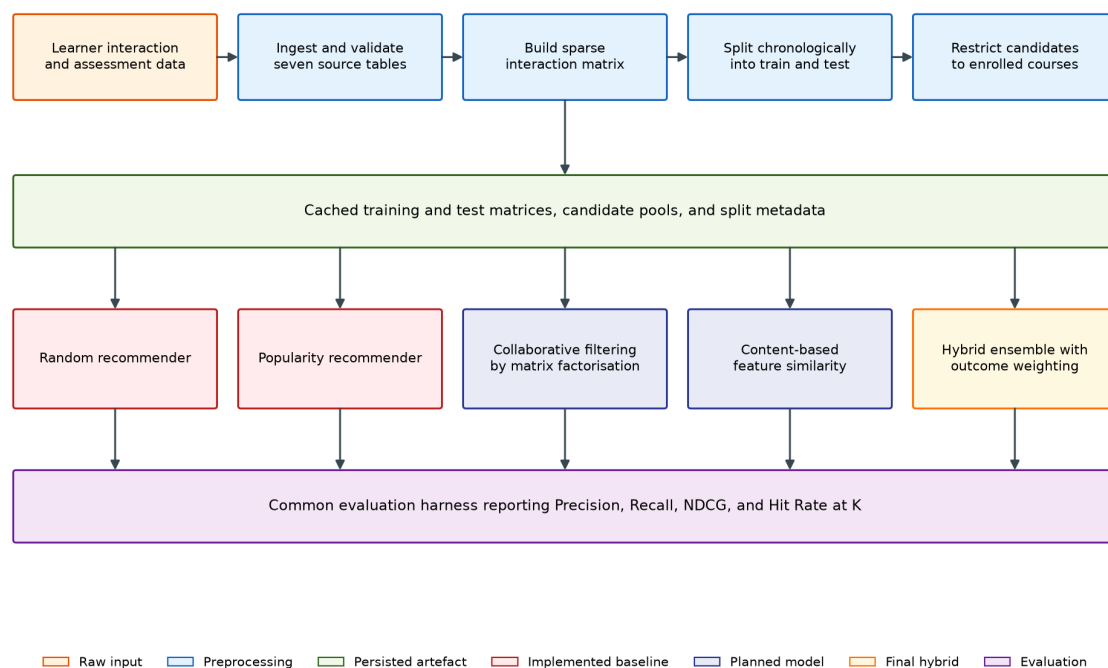


Figure 1: Figure 3.1 — Pipeline architecture: from raw OULAD CSVs through preprocessing, modelling, and evaluation.

The architectural choices behind this layout are deliberate. Separating ingestion from preprocessing means I treat the raw CSVs as immutable inputs; every derived artefact can be reproduced from a clean checkout. Separating preprocessing from modelling means every model consumes the same matrices and can therefore be compared like-for-like. Separating modelling from evaluation means I can add a new model without touching the evaluation harness. My intent is that a marker can pull the repository, run a single command, and reproduce every number reported in this submission.

3.2 Data Design

I use the Open University Learning Analytics Dataset (OULAD; Kuzilek, Hlosta & Zdrahal, 2017), distributed under a CC-BY licence and obtainable from the UCI ML Repository. OULAD is, as far as I am aware, the only public dataset that links *click-level* interaction with *graded assessment outcomes* and *demographic* metadata at the individual learner level, which is what makes it suitable for my outcome-aware evaluation strategy. Table 3.1 summarises the seven CSV tables I use.

Table	Records	Role in pipeline
studentVle	~10.6 M	User-item interactions (sum_click weights)
studentAssessment	~173 K	Outcome signal (assessment score per student per task)
studentInfo	~32 K	Demographics; DEI analysis cohorts
studentRegistration	~32 K	Course registration windows; defines active period
vle	~6.4 K	Items (catalogue) with activity_type taxonomy
assessments	206	Assessment metadata (type, weight, due date)
courses	22	Course/presentation identifiers

Table 3.1 — OULAD tables and their role in the pipeline.

My unit of recommendation is the VLE `id_site` (an individual learning resource). I build the user-item interaction by summing `sum_click` per (`id_student`, `id_site`, `code_module`, `code_presentation`), which produces a sparse matrix of roughly 32K × 6.4K. I build item-level features from `vle` (`activity_type` one-hot encoded; `week_from`, `week_to` normalised) and enrich them from `studentVle` and `studentAssessment` with two behavioural features: the mean assessment score of students who interacted with each item, and the global access count. I compute both of these strictly from the training portion of the temporal split, to avoid leaking future information into the test set — following Ricci et al. (2015), this is essential for honest offline evaluation.

3.3 Recommendation Process — Worked Example

Figure 3.2 walks through the end-to-end transformation for a single learner.

Concretely, the recommender produces a score for each (`student`, `candidate_item`) pair as $s(u, i) = \alpha \cdot s_{\text{CF}}(u, i) + \beta \cdot s_{\text{content}}(u, i) + \gamma \cdot s_{\text{outcome}}(i)$. The three components are the SVD reconstruction, the cosine similarity between the item’s feature vector and the learner’s average accessed-item profile, and the item’s training-set outcome correlation. I tune the weights α, β, γ on a validation fold and return the top- K items the learner has not already accessed. Figure 3.3 shows the decomposition.

Figure 3.2 — Recommendation process for student 592205 (presentation BBB_2013)

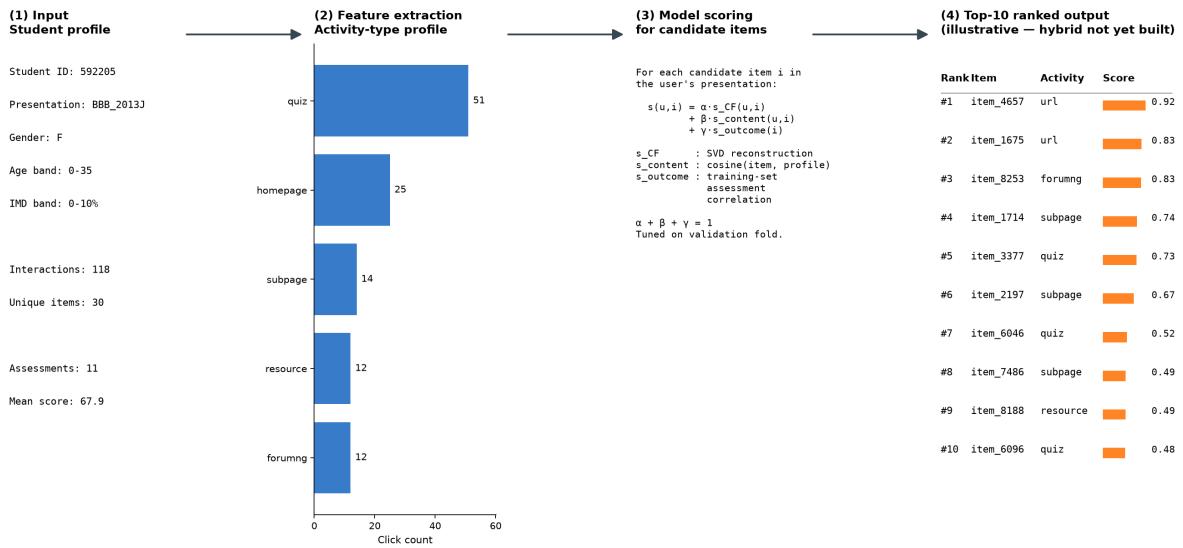


Figure 2: Figure 3.2 — Recommendation process for a real OULAD student: input profile → feature extraction → model scoring → top-10 ranked output. Generated from the prototype against OULAD; the top-10 panel is illustrative since the hybrid model has not yet been trained.

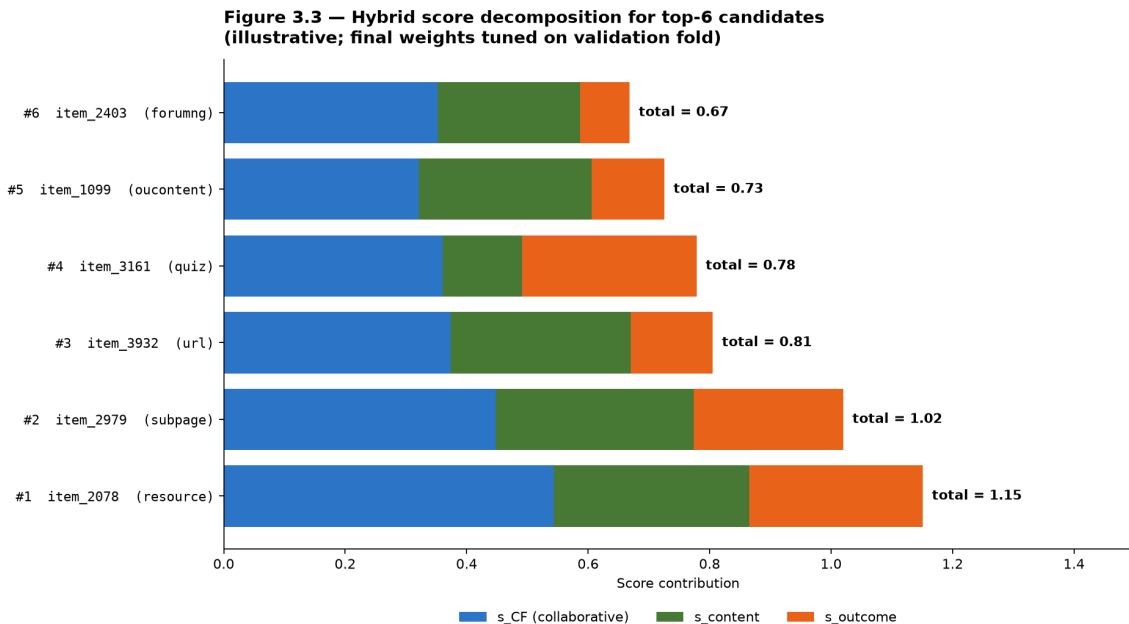


Figure 3: Figure 3.3 — Hybrid score decomposition: per-candidate contributions from collaborative, content, and outcome components, with the total score determining the recommendation rank.

I surface these decompositions in the dashboard so the recommendations are auditable: we can see *why* each item was recommended, not just *that* it was.

3.4 Model Design

I implement five recommenders behind a common Recommender interface, so that every model is evaluated by the same harness on the same split.

1. **Random.** Samples K items uniformly from the catalogue, not from the student's training history. Establishes the floor.
2. **Popularity.** Returns the K most-accessed items globally (training portion only). A surprisingly strong baseline in education, because the most popular VLE items in OULAD tend to be the course's central resources.
3. **Collaborative (SVD).** Truncated SVD on the centred interaction matrix. I plan to tune the number of latent factors $k \in \{20, 50, 100\}$ and regularisation $\lambda \in \{0.01, 0.05, 0.1\}$ by 5-fold CV, following Koren, Bell & Volinsky (2009).
4. **Content-Based.** Cosine similarity over item feature vectors. Each learner's profile is the mean of the feature vectors of items they accessed in training. Items are ranked by similarity to that profile. I include this following Bousbahi & Chorfi (2015), specifically for the cold-start case where SVD has too little interaction signal.
5. **Hybrid.** Weighted ensemble of SVD and content scores, with an optional outcome-weighting term. I will tune the weights by grid search on a validation fold.

The progression is deliberate: each step adds a single concept (popularity $\rightarrow$ personalisation $\rightarrow$ content awareness $\rightarrow$ outcome awareness), so that I can isolate the contribution of each component in the final ablation study.

3.5 Evaluation Strategy

I structure this section around three questions: what I measure, what I compare against, and with what statistical confidence.

What I measure. I compute five metrics for every model.

Metric	Definition	Why included
Precision@10	Fraction of top-10 recommendations that appear in the held-out interaction set	Standard recommender quality
Recall@10	Fraction of held-out interactions captured in top-10	Coverage
NDCG@10	Discounted-cumulative-gain at 10, normalised to ideal ranking	Ranking quality (primary; follows Rendle et al., 2012)
Hit Rate@10	Fraction of users for whom at least one held-out item appears in top-10	User-level success

Metric	Definition	Why included
Outcome-weighted Precision@10	Precision@10 weighted by the mean assessment score of training-set users who accessed each recommended item	Outcome-awareness — my principal contribution

The first four are standard and benchmarked against the prior literature. The fifth is my outcome-aware contribution. It operationalises “learning improvement” in the only way I can offline: by measuring whether recommended items are correlated with higher assessment scores in held-out training data.

Against what. I compare every model against the Random and Popularity baselines, which establish floor and naive-strong performance, and against each preceding model in the progression. The principal hypothesis test of the project is whether the hybrid beats its components.

With what statistical confidence. I evaluate under 5-fold temporal cross-validation. For each fold I split the data chronologically — not randomly — into 80% training and 20% test, advancing the cutoff date across folds. I assess statistical significance of one model over another by paired *t*-test of per-fold metrics with Bonferroni correction across the five metrics. The ablation study disables each component of the hybrid in turn and reports the resulting drop.

The link between this evaluation strategy and the literature I reviewed in Chapter 2 is explicit. Temporal splitting follows Ricci et al. (2015). NDCG as the primary metric follows the implicit-feedback argument of Rendle et al. (2012). Evaluating ranking quality rather than rating prediction is informed by the critique of pure-prediction evaluation that Thai-Nghe et al. (2010) leave unaddressed, and by Wilson et al.’s critique of DKT (2016), which urges caution on any model claiming improvement.

3.6 Ethics, DEI, and Risks

OULAD contains sensitive demographic attributes for every student: gender, age band, region, IMD (Index of Multiple Deprivation) band, and disability flag. I handle these as follows. First, I do not condition recommendations on demographic attributes directly; the model sees only behavioural and content features. Second, I include a *fairness audit* in the evaluation report, breaking per-metric performance down by gender, IMD band, and disability flag, so any group for which the recommender systematically underperforms is visible. This is consistent with the recommendation-system fairness literature and is the way I address the DEI learning objective in concrete, measurable terms. Third, I do not publish or share demographic data beyond aggregate metrics in the report.

I foresee two principal risks going forward. First, the hybrid model might not significantly improve on SVD alone — if so, I will document this as a valid empirical finding rather than a failure of method. Second, sparsity may produce unstable CF metrics for cold-start users, which is the explicit reason I include a content-based component and report cold-start results separately.

4 Feature Prototype

4.1 What the Prototype Implements

The prototype I submit with this preliminary report is a working slice of the system described in Chapter 3. It exercises every stage of the pipeline end-to-end on the full OULAD dataset (10.6

million interactions, 32,593 students, 6,364 VLE items). It has four components.

1. A **data loader** that ingests the seven OULAD CSV tables and exposes them through a single typed container.
2. A **preprocessing module** that builds a sparse user-item interaction matrix from VLE, performs a temporal 80/20 train/test split on the date column, and computes a per-user *course-scoped candidate set* from `studentRegistration`, so that recommenders only propose items from presentations the user is enrolled in.
3. An **evaluation framework** implementing Precision@K, Recall@K, NDCG@K, and Hit Rate@K, with a harness that returns mean per-user scores.
4. Two **baseline recommenders** — Random and Popularity — behind a common `fit / recommend` interface that accepts a candidate-pool argument. A pipeline runner ties everything together: `load → split → fit → evaluate at K in {5, 10, 20} → persist to evaluation/baseline_results.csv`.

Every component the final hybrid system needs is in place. The SVD, content-based, and hybrid recommenders I plan to add will implement the same `Recommender` interface and be evaluated by the same harness on the same split, so the comparisons I make in the final report will be controlled and reproducible. I have written twenty-seven unit tests that cover the metrics, both baselines, candidate-pool restriction, edge cases, and recommender determinism. All pass on a fresh checkout, and the full pipeline runs against OULAD in about three minutes on a laptop.

4.2 Design Decisions and Justification

Four design decisions deserve explicit justification. Each is informed either by the literature I reviewed in Chapter 2 or by a structural property of OULAD I discovered during the prototype build.

Sparse representation throughout. The OULAD interaction matrix is $26,074 \times 6,268$ (after restricting to users with at least one VLE interaction) with 2.24 million non-zero entries across train and test — a density of roughly 1.4%. Dense storage would take around 650 MB of float32; CSR storage takes roughly 30 MB. More importantly, every operation downstream (matrix factorisation, item-item similarity) has an efficient sparse variant. I therefore commit to CSR matrices end-to-end and never densify. This is standard practice in the recommender-system literature (Koren et al., 2009).

Temporal — not random — split. I cut at the 80th percentile of date in `studentVle`, which corresponds to day 172 from course start. Training interactions are those at or before the cutoff; test interactions are those after. This is the temporal protocol argued for by Ricci, Rokach & Shapira (2015): random sampling would let future information leak into training and overestimate performance, particularly for a system whose intended use is sequential (recommending what to study *next*). The cost of temporal splitting is that the test set is biased towards late-course interactions, which I will return to in the final evaluation.

Course-scoped candidate sets. This decision emerged from the first end-to-end pipeline run and turned out to be the most consequential structural property of OULAD for a recommender. Each `id_site` belongs to exactly one (`code_module`, `code_presentation`); the catalogue is partitioned by presentation rather than shared. An average student is enrolled in only 1.1 presentations and therefore has access to a median of 324 items (~5% of the 6,268 total). A naïve global Popularity recommender will mostly suggest items from presentations the user is not enrolled in, which is meaningless whatever metric I report. So I compute per-user candidate sets from `studentRegistration` and have the recommenders restrict their pool to that set. My first run *without* this restriction gave `Popularity@10 = 0.00003` — an order of magnitude worse than Random. Adding course-scoping moved `Popularity@10` to 0.204. The pipeline retains a

--no-course-scoping diagnostic flag so the finding remains reproducible on demand. This is a non-trivial methodological finding in its own right, and it only surfaced because I had the pipeline running end-to-end.

Common interface across recommenders. Both baselines implement `fit(train)` and `recommend(user_row, k, candidates, exclude_seen)` — the standard implicit-feedback formulation (Rendle et al., 2012). I adopted this interface now so that the SVD, content-based, and hybrid models I add later drop into the same harness without changes to the evaluation code.

4.3 Results

The pipeline runs end-to-end against the full OULAD dataset and produces the results in Table 4.1. The evaluation covers 17,562 users — every user with at least one interaction in the held-out test window. All numbers are mean per-user scores; the harness writes them to `evaluation/baseline_results.csv`.

Table 4.1 — Baseline results on OULAD with course-scoped candidates. Per-user means over 17,562 users. Higher is better for all four metrics.

Model	K	Precision@K	Recall@K	NDCG@K	Hit Rate@K
Random	5	0.0712	0.0131	0.0719	0.2872
Random	10	0.0689	0.0253	0.0693	0.4470
Random	20	0.0689	0.0502	0.0731	0.6319
Popularity	5	0.2298	0.0398	0.2342	0.5446
Popularity	10	0.2035	0.0724	0.2171	0.6942
Popularity	20	0.1710	0.1228	0.1998	0.8198

I draw three observations from these numbers. First, Popularity beats Random by roughly a factor of three on Precision and NDCG and by 1.5–2× on Hit Rate. This is the direction I expected (Ricci et al., 2015), and it confirms that the harness produces consistent, comparable numbers across models. Second, Hit Rate@20 reaches 0.82 for Popularity, meaning that for four out of five users at least one of their held-out items appears in the top-20 most popular items within their presentation. That gives me an early sense of how high raw popularity can climb before any personalisation. Third, Precision falls with K for both models — fewer of the top-K items are relevant on average as K grows — while Recall and Hit Rate rise. This is standard ranking-metric behaviour and a useful sanity check.

These numbers set the bar the SVD, content-based, and hybrid models need to clear in the final report.

4.4 Evaluation of the Prototype

Following Rendle et al. (2012) and Ricci et al. (2015), the harness reports Precision@K, Recall@K, NDCG@K, and Hit Rate@K — the standard set for implicit-feedback top-K recommendation. I report all four because no single metric captures recommender quality: Precision can hide poor coverage, and Recall can hide poor ranking. The outcome-weighted Precision metric described in Chapter 3 is *not* yet in the harness. It depends on a feature-engineering step I have not yet built, and it is the principal honest gap between this prototype and the final system.

I chose the two baselines deliberately. Random fixes the floor: any model that fails to beat Random has not learnt anything. Popularity is the strong baseline a useful recommender has to

clear (Thai-Nghe et al., 2010; Bousbahi & Chorfi, 2015) — the most-accessed items in a course tend to be its central resources. I enforce reproducibility throughout: Random is seeded and deterministic; the split and candidate sets are persisted to disk; twenty-seven unit tests guard the metric and baseline code; and one command runs the full pipeline from a fresh checkout.

4.5 Limitations and Planned Improvements

The prototype has three principal limitations, all of them deliberate scoping decisions for this submission rather than oversights.

First, **only the baselines are implemented**. The SVD, content-based, and hybrid models are not in this submission. They will use the same interfaces and the same harness, so they will integrate without any architectural change.

Second, **outcome-aware evaluation is not yet in the harness**. Outcome-weighted Precision needs per-item mean assessment scores from the training portion of `studentAssessment`, which in turn needs the feature engineering step I have not built. I have sketched the schema in the design chapter.

Third, **the temporal split is single-fold**. Five-fold temporal cross-validation, as I argued in Chapter 3, is the protocol for the final evaluation. A single fold is enough to show the harness produces sensible numbers; the multi-fold harness is a small extension and is scheduled for the next submission.

The path from prototype to final system is short and well-defined. The remaining work falls into these blocks: SVD collaborative filtering, feature-based content filtering, hybrid ensemble with ablation, FastAPI service and Streamlit dashboard, and final cross-validation and significance testing. The plan is feasible because the technically novel steps each use library implementations with well-known performance characteristics — my contribution is in the combination and evaluation, not in implementing new algorithms from scratch.

References

1. Bousbahi, F. and Chorfi, H., 2015. MOOC-Rec: A case based recommender system for MOOCs. *Procedia — Social and Behavioral Sciences*, 195, pp.1813–1822.
2. Corbett, A.T. and Anderson, J.R., 1995. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4), pp.253–278.
3. Koren, Y., Bell, R. and Volinsky, C., 2009. Matrix factorization techniques for recommender systems. *IEEE Computer*, 42(8), pp.30–37.
4. Kuzilek, J., Hlosta, M. and Zdrahal, Z., 2017. Open University Learning Analytics dataset. *Scientific Data*, 4(1), pp.1–8.
5. Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L.J. and Sohl-Dickstein, J., 2015. Deep knowledge tracing. *Advances in Neural Information Processing Systems*, 28, pp.505–513.
6. Rendle, S., Freudenthaler, C., Gantner, Z. and Schmidt-Thieme, L., 2012. BPR: Bayesian personalized ranking from implicit feedback. *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence (UAI)*, pp.452–461.
7. Ricci, F., Rokach, L. and Shapira, B. (eds.), 2015. *Recommender Systems Handbook*, 2nd edn. Springer, New York.

8. Thai-Nghe, N., Drumond, L., Krohn-Grimberghe, A. and Schmidt-Thieme, L., 2010. Recommender system for predicting student performance. *Procedia Computer Science*, 1(2), pp.2811–2819.
9. Wilson, K.H., Karklin, Y., Han, B. and Ekanadham, C., 2016. Back to the basics: Bayesian extensions of IRT outperform neural networks for proficiency estimation. *Proceedings of the 9th International Conference on Educational Data Mining*, pp.539–544. arXiv:1604.02336.